

FINAL PROJECT REPORT

SEMESTER 1, ACADEMIC YEAR: 2025-2026

CT312H: MOBILE PROGRAMMING

- Project/Application name: Biblio - Task and habits management
- GitHub links: <https://github.com/25-26Sem1-Courses/ct312hm01-project-Wang6006.git>
- Demo link: <https://youtu.be/f27f1I8XbSw>
- Student ID 1: B2206006
- Student Name 1: Bui Minh Quang
- Student ID 2: B2206024
- Student Name 2: Huynh Thi Thuy Vy
- Class/Group Number: CT312HM01/15

1 Introduction

1.1 Project Overview

Biblio is a comprehensive personal productivity management mobile application built with Flutter. The application combines two powerful productivity methodologies: task management and habit tracking. Users can organize their daily tasks through a category-based system, track long-term habits, and analyze their productivity through statistical visualizations.

1.2 Task Assignment

Member	Tasks
Bui Minh Quang	Setting, Notification, Deployment
Huynh Thi Thuy Vy	Task Management, Habit Management

2 Details of Implemented Features

2.1 Task Management System

Database Tables:

```
CREATE TABLE categories(  
    id          INTEGER          PRIMARY KEY  
    AUTOINCREMENT,  
    name TEXT NOT NULL,  
    "index" INTEGER NOT NULL DEFAULT 0,  
    color TEXT NOT NULL DEFAULT 'gray'  
)  
  
CREATE TABLE tasks(  
    id          INTEGER          PRIMARY KEY  
    AUTOINCREMENT,  
    categoryId INTEGER NOT NULL,  
    title TEXT NOT NULL,
```

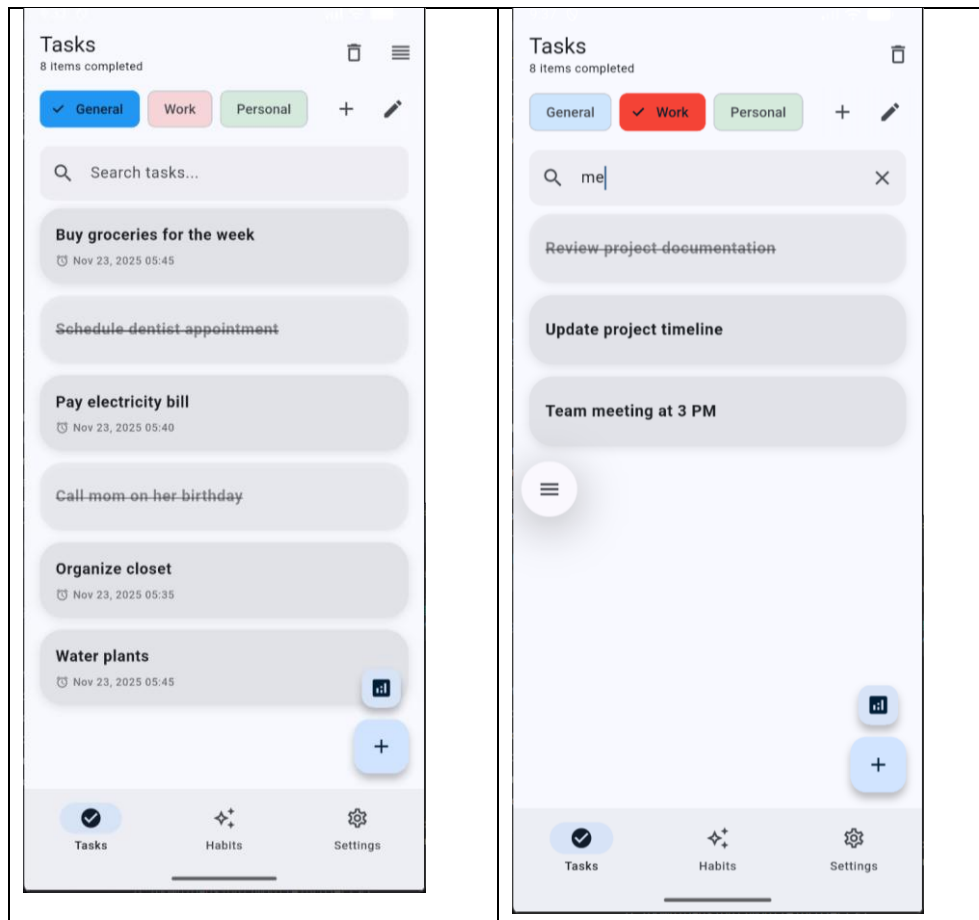
```
"index" INTEGER NOT NULL DEFAULT 0,  
status INTEGER NOT NULL DEFAULT 0,  
reminder TEXT,  
        FOREIGN KEY (categoryId) REFERENCES  
categories(id) ON DELETE CASCADE  
)  
  
CREATE INDEX idx_tasks_categoryId ON  
tasks(categoryId)'
```

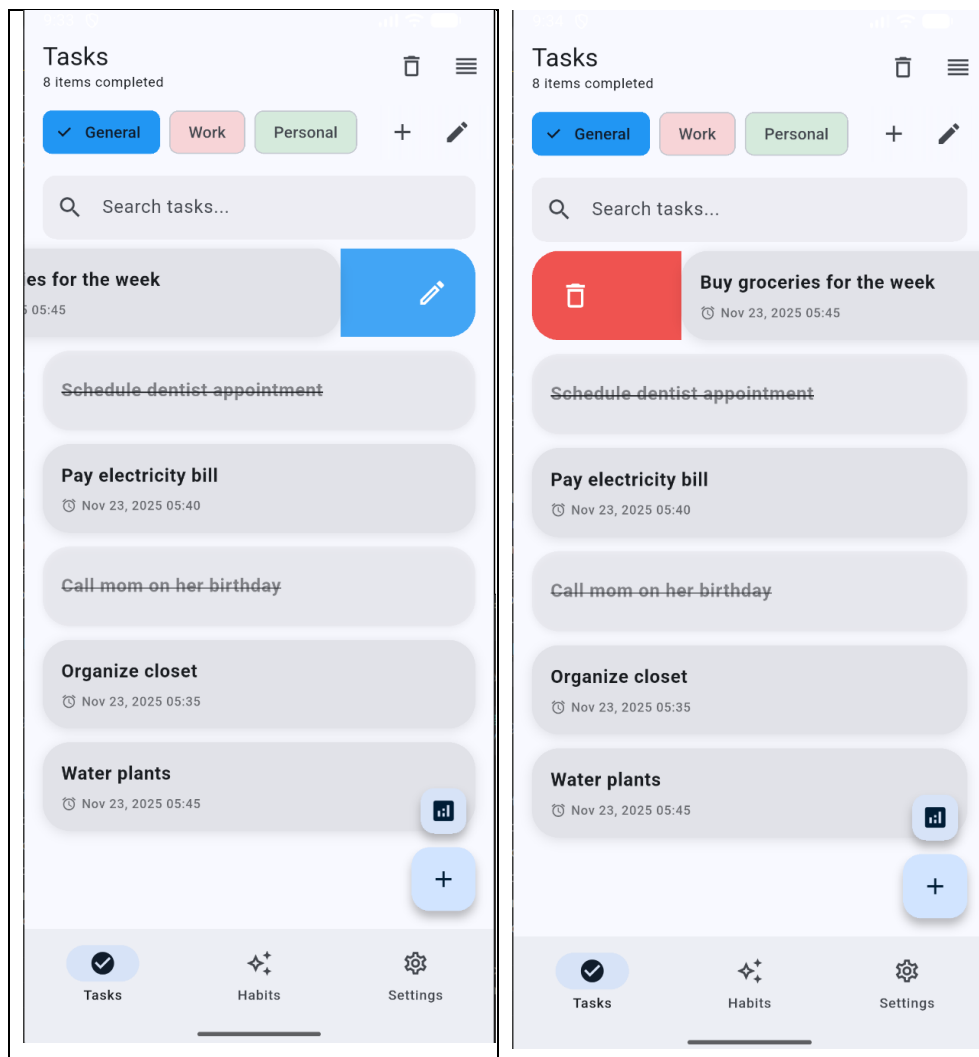
2.1.1 Task List Screen

Description

The Task List Screen serves as the primary interface for task management. Users can view tasks grouped by categories, toggle completion status, edit tasks, and reorder them by priority. The screen features a floating action button for quick task creation and horizontal category chips for switching between task groups.

Screenshots





Implementation Details

- Key Widgets Used:
 - `ReorderableView.builder`: Enables drag-and-drop list reordering with custom item positioning
 - `ReorderableDragStartListener`: Provides precise control over when drag operation begins, allowing custom drag handles outside the item card
 - `AnimatedVisibility`: Smooth fade-in/fade-out transitions when showing/hiding UI elements
- Libraries/Plugins:

- provider (v6.1.1): Used for reactive state management. The `TasksManager` class extends `ChangeNotifier` and notifies UI widgets when task data changes, triggering automatic rebuilds without manual `setState` calls.
- intl (v0.18.1): Used for date/time formatting in reminder displays, converting `DateTime` objects to readable format like "Jan 15, 2025 14:30".
- State Management and Architecture: Provider pattern with `ChangeNotifier`: The `TasksManager` class holds all task-related state (task lists, current category, loading status) and provides methods to modify this state. When data changes, it calls `notifyListeners()` which signals all listening widgets to rebuild. The UI layer accesses this state through `Consumer` widgets or `context.read()/watch()` methods. Architecture follows three layers: Presentation (UI widgets) → Business Logic (`TasksManager`) → Data Access (`DatabaseHelper`).

Data Storage:

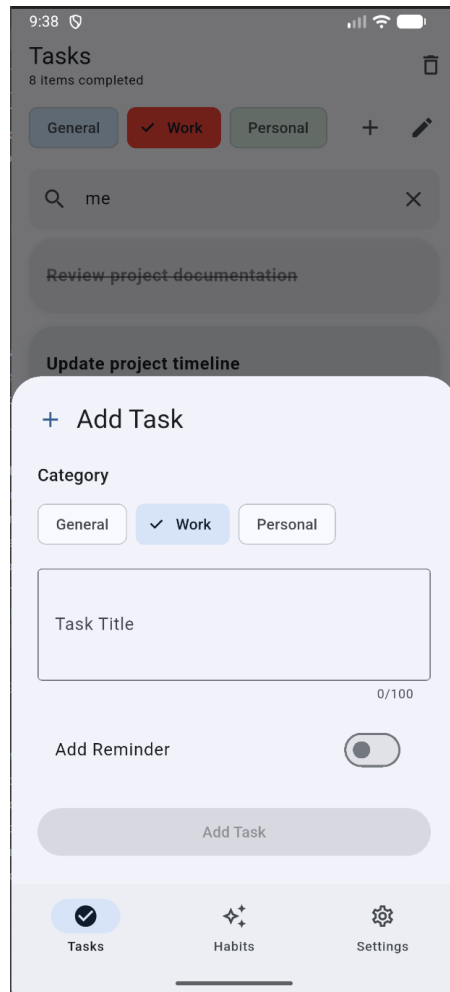
- Read operations:
 - Loads all categories and tasks on screen initialization
 - Queries tasks filtered by selected category
 - Retrieves task counts for statistics
- Store operations:
 - Inserts new tasks with auto-incrementing IDs
 - Updates task status when toggled (pending ↔ completed)
 - Updates task index values when reordered
 - Deletes individual tasks or batch deletes completed tasks

2.1.2 Task Add/Edit

Description

A modal bottom sheet form for creating new tasks or editing existing ones.

Screenshots



Implementation Details

- Key Widgets Used:
 - showModalBottomSheet: Flutter API for displaying modal sheets that slide up from bottom

- `showDatePicker` & `showTimePicker`: Material Design date and time picker dialogs with calendar and clock interfaces
- `MediaQuery.viewInsets.bottom`: Detects keyboard height to dynamically resize sheet and prevent content being hidden
- Libraries/Plugins:
 - `intl` (v0.18.1): Used for formatting the selected reminder date/time into human-readable format "MMM d, y HH:mm" displayed below the reminder toggle switch.
- State Management and Architecture: Uses local `StatefulWidget` state for form data (title, category selection, reminder) to avoid polluting global state during editing. Only commits changes to global `TasksManager` state when user taps Save button. The sheet receives `Task` object as parameter, user edits it locally, then returns modified `Task` via `Navigator.pop()`. Parent screen receives result and calls `TasksManager.addTask()` or `updateTask()` which updates database and notifies listeners.

Data Storage:

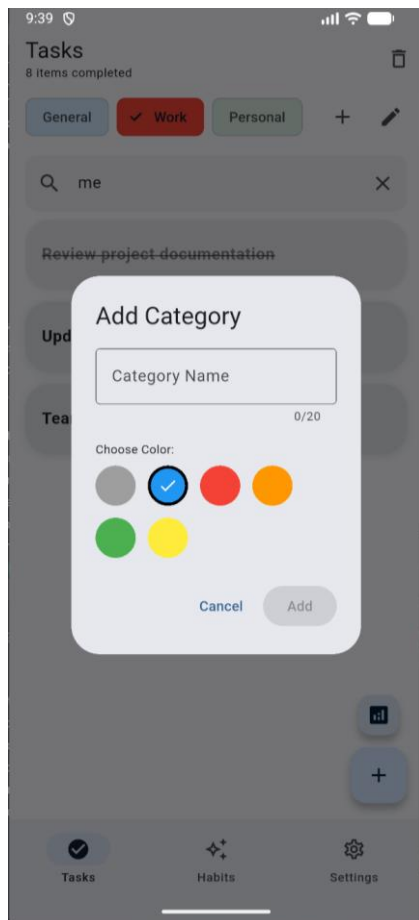
- Read operations:
 - Receives existing `Task` object when editing (for pre-filling form fields)
 - Loads category list from `TasksManager` for category selector
- Store operations:
 - Inserts new tasks with auto-incrementing IDs
 - Updates task status when toggled (pending ↔ completed)
 - Updates task index values when reordered
 - Deletes individual tasks or batch deletes completed tasks

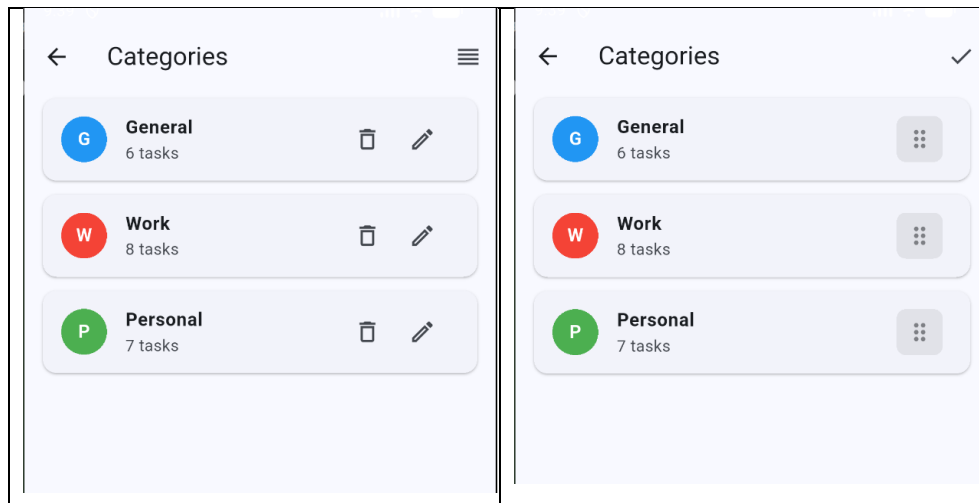
2.1.3 Category Management Screen

Description

Dedicated screen for managing task categories. Supports full CRUD operations: create category with name and color, edit category properties, delete category (removes associated tasks via CASCADE), and reorder categories via drag-and-drop.

Screenshots





Implementation Details

- Key Widgets Used:
 - ReorderableListView: Same as Task List but for categories
 - Custom color picker using Wrap + GestureDetector: Creates grid of circular color buttons (40×40px) that change appearance when selected (black border + checkmark icon)
- Libraries/Plugins:
 - provider (v6.1.1): Used for accessing TasksManager to perform CRUD operations on categories and trigger UI updates.
- State Management and Architecture: uses global TasksManager state for category list. Combines with local state for UI controls (edit mode toggle, dialog visibility). When user creates/updates/deletes/reorders categories, changes are saved to database through TasksManager which then calls notifyListeners() to update all screens displaying categories (including Task List screen's category selector).

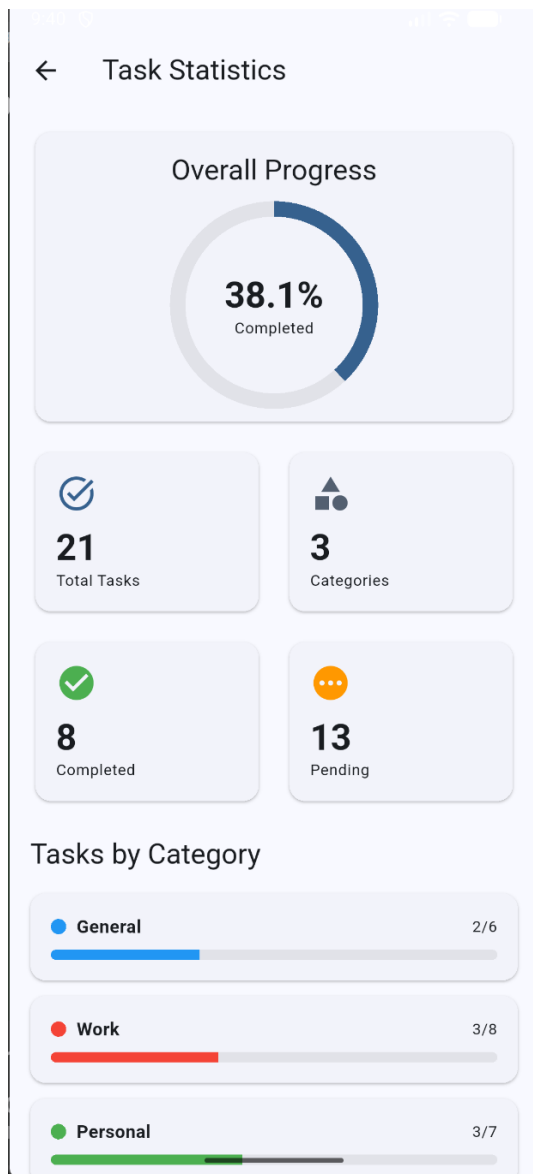
- Data Storage:
 - Read operations:
 - Loads all categories with associated task counts
 - Queries categories ordered by index field for proper display sequence
 - Store operations:
 - Inserts new category with auto-calculated index (max existing index + 1), default color is blue
 - Modifies category name and/or color in database
 - Removes category (CASCADE automatically deletes all tasks in that category)
 - Batch updates index field for all affected categories when user drags to new position

2.1.4 Task Statistics Screen

Description

Analytics dashboard providing visual insights into task completion and productivity. Displays overall completion percentage using circular progress indicator, per-category breakdown with linear progress bars, and key metrics cards (total tasks, completed, pending, category count).

Screenshots



Implementation Details

- Key Widgets Used:
 - Stack with centered Text overlay: Displays percentage text in center of CircularProgressIndicator
 - Custom `_StatCard` component: Reusable metric display combining Icon + value + label in structured Card layout

- Libraries/Plugins:
 - provider (v6.1.1): Used for reactive data consumption from TasksManager. Screen only reads state, does not modify it.
- State Management and Architecture: uses read-only access to TasksManager global state. Screen calculates statistics in real-time from task data: completion rate = $(\text{completed count} \div \text{total count}) \times 100$, per-category progress = $\text{completed tasks in category} \div \text{total tasks in category}$. All calculations performed in-memory without additional database queries for optimal performance. Consumer widget ensures screen rebuilds automatically when tasks are added/completed/deleted.
- Data Storage:
 - Read operations:
 - Retrieves complete task list from TasksManager
 - Filters tasks by status to count completed vs pending
 - Groups tasks by category for per-category statistics
 - Accesses category list with colors for visual display
 - Store operations: This is a read-only analytics screen.

2.2 Habit Management System

Database Tables:

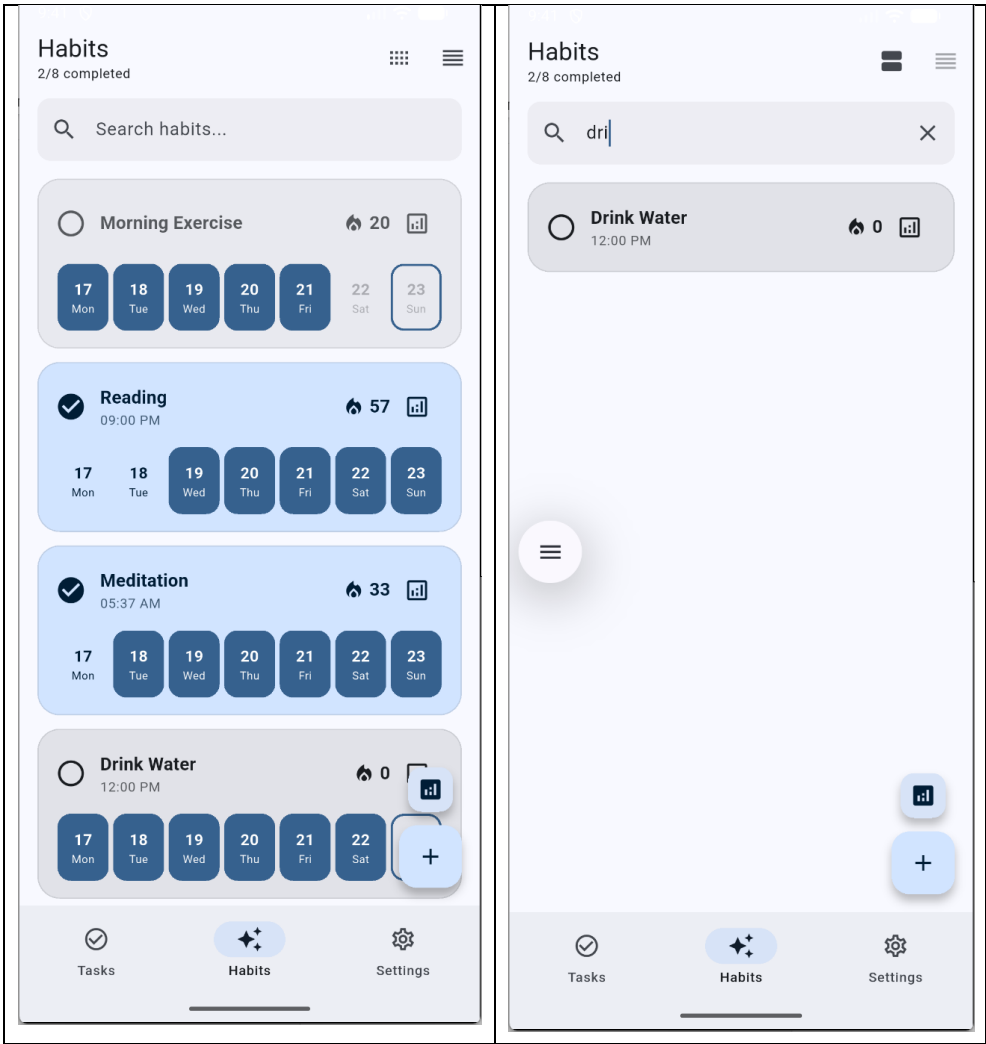
```
CREATE TABLE habit_status(  
    id INTEGER PRIMARY KEY  
    AUTOINCREMENT,  
    habitId INTEGER NOT NULL,  
    date TEXT NOT NULL,  
    FOREIGN KEY (habitId) REFERENCES habits(id)  
    ON DELETE CASCADE,  
    UNIQUE(habitId, date)  
)  
  
CREATE TABLE habits(  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    title TEXT NOT NULL,  
    description TEXT NOT NULL,  
    "index" INTEGER NOT NULL DEFAULT 0,  
    days TEXT NOT NULL DEFAULT '1,2,3,4,5,6,7',  
    time TEXT NOT NULL,  
    reminder INTEGER NOT NULL DEFAULT 0,  
    frequency TEXT NOT NULL DEFAULT 'weekly',  
    frequencyDays INTEGER NOT NULL DEFAULT 7,  
    createdAt TEXT NOT NULL DEFAULT  
(datetime('now'))  
)  
  
CREATE INDEX idx_habit_status_habitId ON  
habit_status(habitId);  
CREATE INDEX idx_habit_status_date ON  
habit_status(date);
```

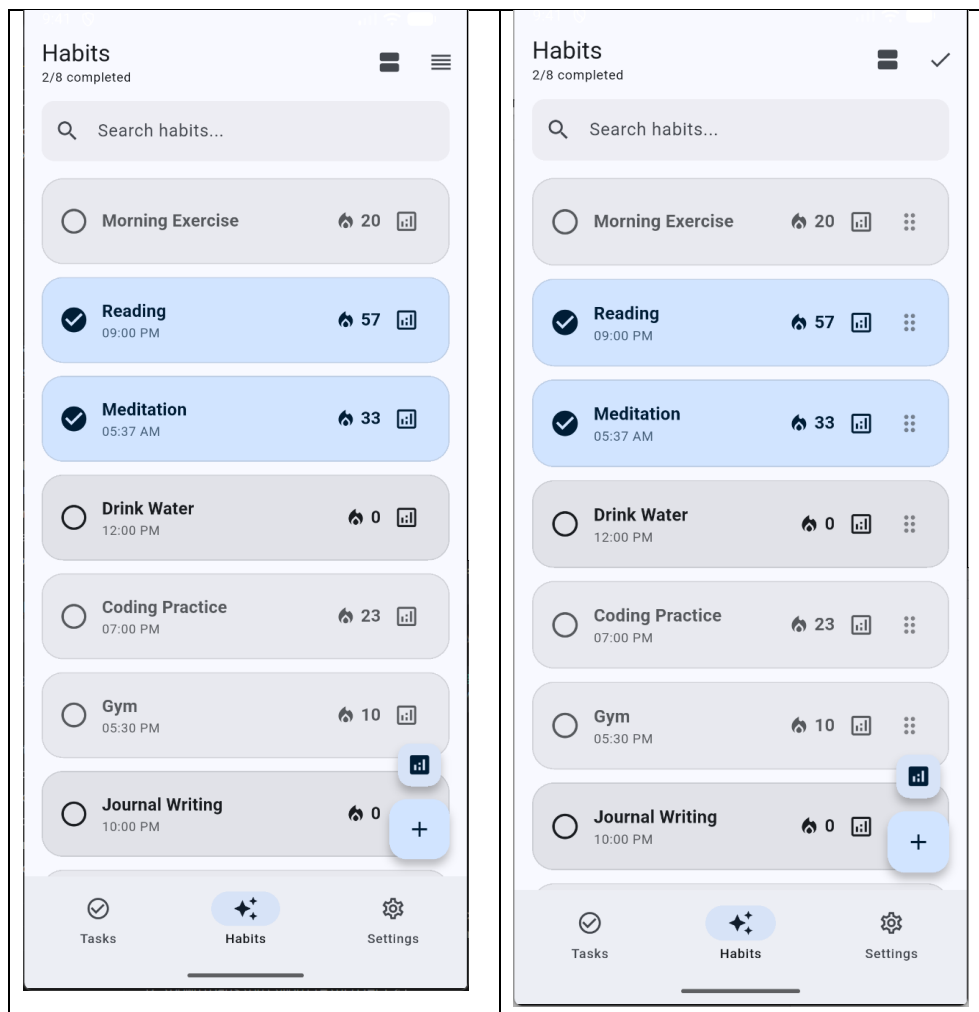
2.2.1 Habits List Screen

Description

The Habits List Screen displays all user habits in a card-based layout with weekly progress visualization. Each habit card shows completion status, current streak, and a 7-day mini calendar. Users can toggle habit completion by tapping cards (only on scheduled days), access detailed analytics, and reorder habits via drag-and-drop.

Screenshots





Implementation Details

- Key Widgets Used:

- `ReorderableListView.builder`: Enables drag-and-drop habit reordering. Users can hold and drag habit cards to change their display order, which persists across app sessions.
- `AnimatedSwitcher` with `ScaleTransition`: Creates smooth visual feedback when toggling habit completion status. The check icon scales up/in when completed, providing satisfying interaction feedback.

- TweenAnimationBuilder: Animates color transitions for completed vs. pending states. Cards smoothly transition between primary container color (completed) and surface color (pending).
 - ListView.builder (horizontal): Renders the 7-day week view inside each habit card, showing completion status for the current week with date and day labels.
- Libraries/Plugins:
 - provider (v6.1.1): State management solution connecting UI to HabitsManager. Consumer widgets listen to habit data changes and rebuild automatically when habits are added, completed, or reordered.
 - intl (v0.18.1): Formats reminder times to locale-specific 12/24-hour format (e.g., "02:30 PM" or "14:30"). Also formats dates in the week view with abbreviated day names (Mon, Tue, etc.).
 - table_calendar (v3.0.9): Powers the weekly calendar visualization in non-compact view mode, displaying the current week with proper day-of-week alignment and completion indicators.
- State Management and Architecture: Provider pattern with ChangeNotifier. The HabitsManager class maintains a Map<Habit, List<HabitStatus>> structure that pairs each habit with its completion history. The UI accesses this through Consumer widgets that rebuild when data changes. Architecture follows three layers:
 - Presentation Layer: UI widgets (HabitCard, HabitsListScreen) display data and capture user interactions

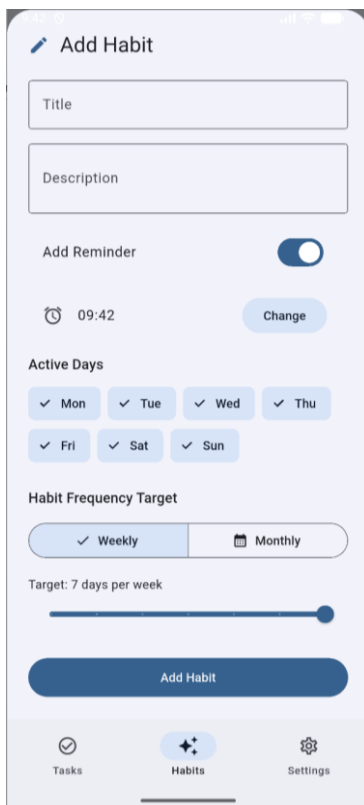
- Business Logic Layer: HabitsManager processes habit operations (add, update, delete, toggle status) and calculates streaks/statistics
- Data Access Layer: HabitDatabaseHelper handles SQLite operations with optimized queries and foreign key constraints
- Data Storage:
 - Read operations:
 - Loads all habits ordered by index on screen initialization
 - Retrieves all habit status records to build completion history map
 - Queries today's completed habits to show completion count
 - Calculates current and best streaks by analyzing status dates
 - Store operations:
 - Inserts new habits with auto-calculated index ($\text{max} + 1$)
 - Updates habit properties (title, description, reminder settings, active days)
 - Inserts habit_status records with UNIQUE constraint to prevent duplicate completions
 - Deletes habit_status records when user un-completes a habit
 - Batch updates habit index values when reordered via drag-and-drop

2.2.2 Habit Add/Edit

Description

A modal bottom sheet form for creating new habits or editing existing ones. Users can set habit title, description, reminder time, and select which days of the week the habit is active.

Screenshots



Implementation Details

- Key Widgets Used:
 - showModalBottomSheet with isScrollControlled: true: Flutter API for displaying modal sheets that expand to full height when keyboard appears, preventing content overlap.
 - showTimePicker: Material Design time picker dialog. Displays 12/24-hour clock based on system settings for selecting habit reminder time.

- Libraries/Plugins:
 - intl (v0.18.1): Formats the selected reminder time in the display label below the time picker button, showing "03:45 PM" or "15:45" based on locale.
 - provider (v6.1.1): Used to call HabitsManager methods (addHabit/updateHabit) when form is submitted.
- State Management and Architecture: Uses local StatefulWidget state (`_HabitUpsertSheetState`) for form inputs. This keeps draft changes isolated from global state until user confirms. The sheet accepts an optional `Habit?` parameter - if provided, form pre-fills with existing values (edit mode). If null, form starts empty (add mode).
- Data Storage:
 - Read operations:
 - Receives existing Habit object when editing (pre-fills form fields)
 - No direct database reads (data passed via constructor)
 - Store operations:
 - Inserts new habits with validated title/description, selected weekdays (stored as comma-separated string "1,2,5" for Mon,Tue,Fri), reminder time, and auto-calculated index
 - Updates existing habit with modified properties while preserving ID and index

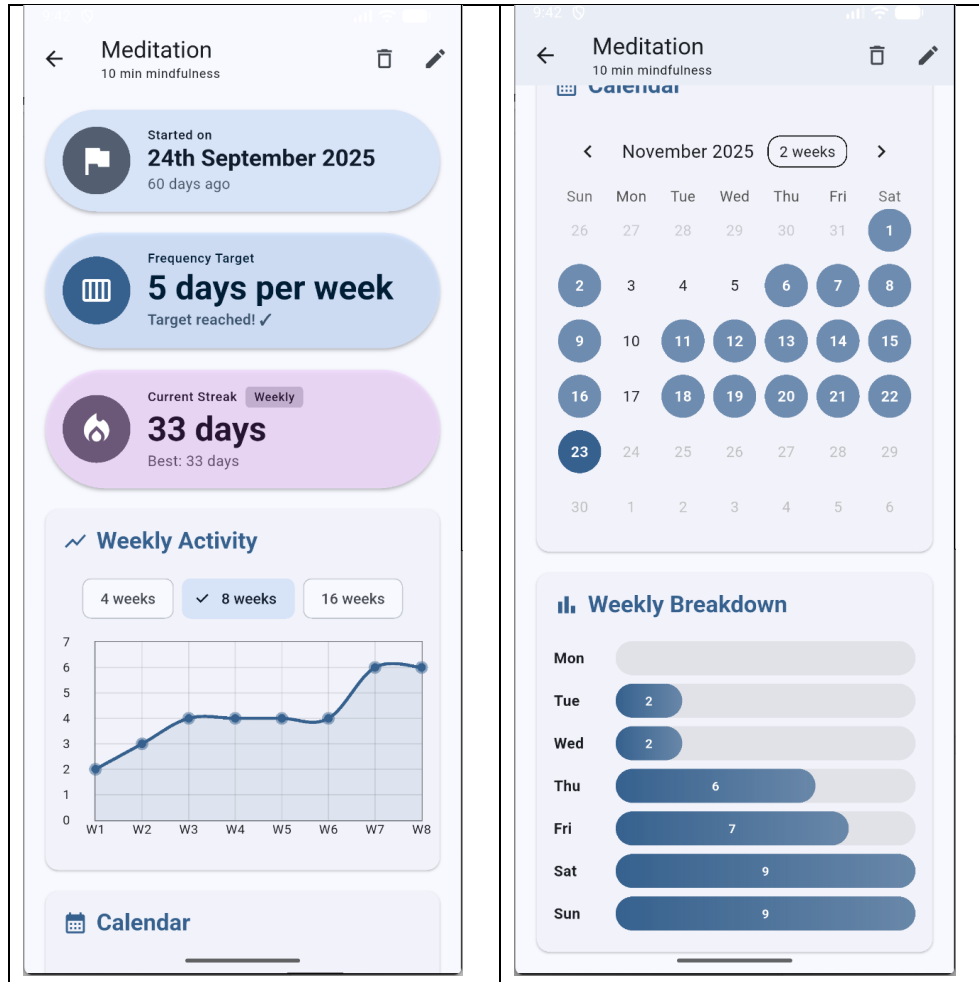
2.2.3 Habit Analytics Screen

Description

Detailed analytics dashboard for individual habits. Displays habit start date, current/best streak with animated fire icon, weekly activity line chart,

interactive calendar for toggling completion on past dates, and weekday breakdown bar chart.

Screenshots



Implementation Details

- Key Widgets Used:
 - TableCalendar (from table_calendar package): Interactive monthly calendar with custom day builders. Completed days show filled circles, invalid days (not in habit's active days) appear dimmed, and users can tap dates to toggle completion.

- LineChart (from fl_chart package): Displays weekly completion trend over 4/8/16 weeks. Shows completion count per week (0-7 range) with smooth curved lines, dots on data points, and gradient fill below the line.
- RotationTransition with AnimationController: Animates the fire icon in streak card. When current streak equals best streak, icon rotates continuously to celebrate achievement.
- Libraries/Plugins:
 - table_calendar (v3.0.9): Provides full-featured calendar widget with custom styling, day builders, and gesture handling for date selection.
 - fl_chart (v0.66.0): Charting library for line chart visualization. Handles touch interactions, gridlines, axis labels, and smooth curve interpolation.
 - provider (v6.1.1): Accesses HabitsManager to load habit data, calculate streaks, and update completion status.
 - intl (v0.18.1): Formats dates in calendar and start date display with ordinal suffixes (1st, 2nd, 3rd, 21st).
- State Management and Architecture: Uses read-mostly access to HabitsManager global state with local state for UI controls (calendar focused day, selected day, chart time period). The Consumer<HabitsManager> widget provides reactive access to habit data.
- Data Storage:
 - Read operations:
 - Retrieves habit details (title, description, active days, start date)
 - Loads all habit_status records for this habit, sorted by date DESC
 - Calculates current and best streaks from status date sequences

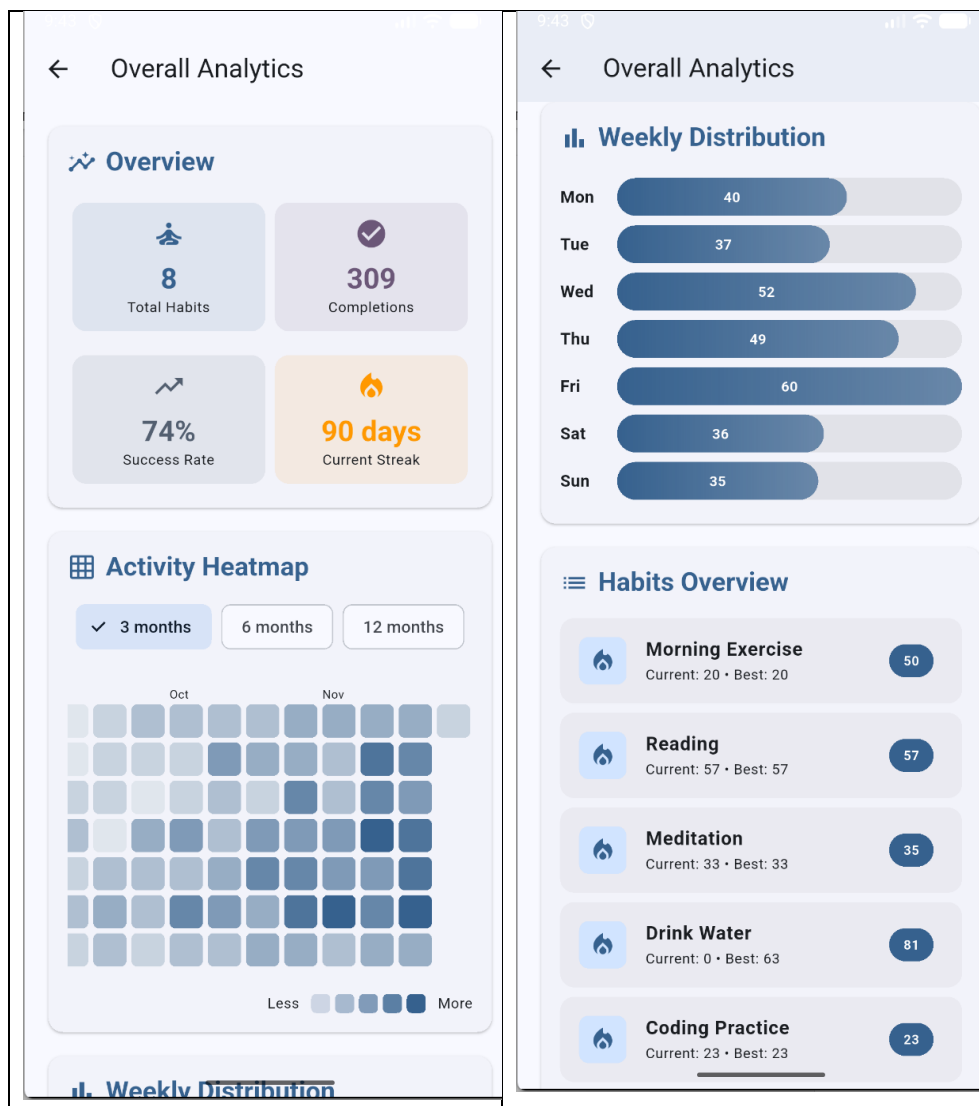
- Queries completion status for each calendar day to render properly
- Store operations:
 - Inserts `habit_status` records when user taps incomplete dates on calendar
 - Deletes `habit_status` records when user taps completed dates (toggle off)
 - Updates habit properties when user opens edit sheet from app bar

2.2.4 Overall Analytics Screen

Description

Consolidated analytics dashboard showing statistics across all habits. Features activity heatmap (365-day grid with color intensity based on completion count), weekly distribution bar chart (which weekdays user is most productive), and per-habit overview list showing total completions and streak stats.

Screenshots



Implementation Details

- Key Widgets Used:
 - HeatMap (from flutter_heatmap_calendar package): Displays year-long grid of completion activity. Each cell represents one day, colored by completion count (darker = more habits completed). Provides quick visual overview of consistency patterns.
 - Custom bar chart using Stack + FractionallySizedBox: Creates horizontal progress bars for weekday distribution. Each bar's width represents percentage of total completions on that weekday.
 - AnimatedFractionallySizedBox: Animates bar width changes with smooth easing curve when data updates.
- Libraries/Plugins:
 - flutter_heatmap_calendar (v1.0.5): Provides GitHub-style contribution heatmap. Accepts Map<DateTime, int> where values determine color intensity.
 - provider (v6.1.1): Reads HabitsManager to aggregate data across all habits for global statistics.
- State Management and Architecture: Read-only screen consuming HabitsManager state. All calculations performed in-memory from existing habit/status data:
 - Heatmap data: Iterates all habit statuses, groups by date, counts completions per day
 - Weekday distribution: Maps status dates to weekdays (1-7), counts occurrences
 - Per-habit stats: For each habit, retrieves status count and calculates streaks
- Data Storage:
 - Read operations:

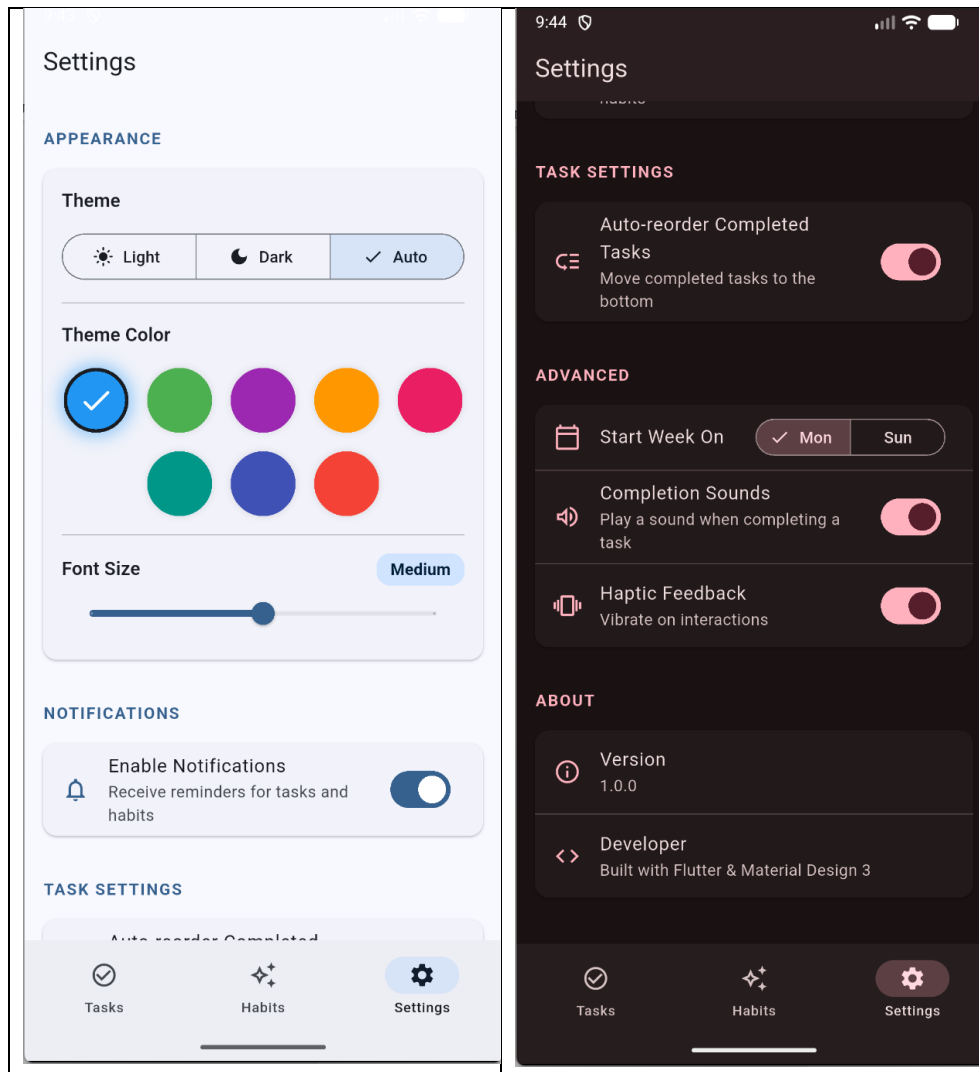
- Aggregates all habit_status records across all habits
- Groups statuses by date to build heatmap (365 days)
- Groups statuses by weekday (1=Monday, 7=Sunday) for distribution chart
- Retrieves habit list with associated status counts
- Calculates current and best streaks for each habit
- Store operations: No write operations (read-only analytics screen)

2.3 Settings Screen

Description

The Settings Screen serves as a centralized configuration hub for customizing the application experience. It provides comprehensive controls organized into logical sections: Appearance (theme mode, color scheme, font size), Notifications (enable/disable reminders), Task Settings (auto-reorder behavior), Advanced (start day, sound effects, haptic feedback), and About (version information).

Screenshots



Implementation Details

Key Widgets Used

- `SegmentedButton<ThemeMode>`: Modern Material 3 widget for mutually exclusive selection. Used for theme mode (Light/Dark/Auto) with icon+label segments.
- Wrap with `GestureDetector`: Creates a responsive grid layout for color scheme picker. Wrap widget automatically flows color circles to new rows based on available width. Each circle is wrapped in `GestureDetector` for tap handling.

- `AnimatedContainer`: Provides smooth transitions when color selection changes. Animates border width (0→3px), shadow intensity, and shows check icon with 200ms duration using `easeInOut` curve.
- `SliderTheme` with `Slider`: Custom-styled slider for font size selection. `SliderTheme` configures thumb size, track height, and value indicator appearance. `Slider` has 3 discrete positions (divisions: 2) for Small/Medium/Large.
- `SwitchListTile`: Material Design list item combining switch, title, subtitle, and leading icon. Used for boolean settings like notifications, auto-reorder, sound effects, and haptics. Provides consistent layout and touch target size (48dp minimum).

Libraries/Plugins

- `provider` (v6.1.1): State management solution connecting UI to three manager classes:
 - `SettingsManager`: Stores user preferences (font size, notifications, sounds, haptics)
 - `ThemeManager`: Controls theme mode (Light/Dark/Auto) and reacts to system theme changes
 - `TasksManager`: Manages task-specific settings (auto-reorder preference)

Uses `context.watch<T>()` for reactive listening and `context.read<T>()` for one-time access in callbacks. When any manager calls `notifyListeners()`, dependent widgets rebuild automatically.

- `flutter_local_notifications` (v17.2.3): Handles notification scheduling and cancellation. The `NotificationService` class wraps this plugin, providing methods like `cancelAllNotifications()` called when user disables notifications. Requires platform-specific configuration in `AndroidManifest.xml` and `Info.plist`.

State Management and Architecture

The screen uses a multi-provider architecture with three separate managers for separation of concerns:

`SettingsManager` (`ChangeNotifier`):

- Manages appearance settings: font size (0.0-2.0 step), theme color (enum of 8 colors)
- Manages behavior settings: start day (Monday/Sunday), sound effects (bool), haptics (bool)
- Manages notification preference: all notifications enabled (bool)
- Provides computed properties: `fontSizeMultiplier` (0.9/1.0/1.25), `fontSizeLabel` ("Small"/"Medium"/"Large")

ThemeManager (ChangeNotifier):

- Controls theme mode: Light, Dark, or System (follows OS preference)
- Stores selected seed color for Material 3 color scheme generation
- Notifies listeners when theme changes to trigger app-wide theme rebuild

TasksManager (ChangeNotifier):

- Manages task-specific preference: `reorderTasksOnComplete` (bool)
- When enabled, automatically moves completed tasks to bottom of list
- This setting persists across app restarts